

Exercise 1: Overview and Setup

Configuration File (application.properties)

```
spring.application.name=EmployeeManagementSystem

spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1

spring.datasource.driverClassName=org.h2.Driver

spring.datasource.username=sa

spring.datasource.password=password

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

spring.jpa.hibernate.ddl-auto=update
```

Project Bootstrap File (EmployeeManagementSystemApplication.java)

```
package com.example.ems;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class EmployeeManagementSystemApplication {

    public static void main(String[] args) {

        SpringApplication.run(EmployeeManagementSystemApplication.class, args);

    }

}
```

Exercise 2: Creating Entities

Department Entity (Department.java)

```
package com.example.ems.model;

import java.util.ArrayList;

import java.util.List;

import javax.persistence.CascadeType;

import javax.persistence.Column;

import javax.persistence.Entity;

import javax.persistence.FetchType;

import javax.persistence.GeneratedValue;

import javax.persistence.GenerationType;

import javax.persistence.Id;

import javax.persistence.OneToOne;

import javax.persistence.Table;

import lombok.Getter;

import lombok.NoArgsConstructor;

import lombok.Setter;

@Entity

@Table(name = "departments")

@Getter

@Setter

@NoArgsConstructor

public class Department {
```

```

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

@Column(name = "department_id")

private Long id;


@Column(name = "department_name", nullable = false)

private String name;


@OneToMany(mappedBy = "department", cascade = CascadeType.ALL, fetch =
FetchType.LAZY)

private List<Employee> employees = new ArrayList<>();
}

```

Employee Entity (Employee.java)

```

package com.example.ems.model;


import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.FetchType;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.ManyToOne;

```

```
import javax.persistence.Table;

import lombok.Getter;

import lombok.NoArgsConstructor;

import lombok.Setter;
```

```
@Entity
```

```
@Table(name = "employees")
```

```
@Getter
```

```
@Setter
```

```
@NoArgsConstructor
```

```
public class Employee {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    @Column(name = "employee_id")
```

```
    private Long id;
```

```
    @Column(name = "employee_name", nullable = false)
```

```
    private String name;
```

```
    @Column(name = "employee_email", nullable = false, unique = true)
```

```
    private String email;
```

```
    @ManyToOne(fetch = FetchType.LAZY)
```

```
    @JoinColumn(name = "department_id")
```

```
private Department department;  
}
```

Exercise 3: Creating Repositories

Department Repository (DepartmentRepository.java)

```
package com.example.ems.repository;  
  
import com.example.ems.model.Department;  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;  
  
@Repository  
public interface DepartmentRepository extends JpaRepository<Department,  
Long> {  
    Department findByName(String name);  
}
```

Employee Repository (EmployeeRepository.java)

```
package com.example.ems.repository;  
  
import com.example.ems.model.Employee;  
import java.util.List;  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;
```

@Repository

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {  
    List<Employee> findByNameContaining(String keyword);  
    Employee findByEmail(String email);  
}
```

Exercise 4: Implementing CRUD Operations

Department Controller (DepartmentController.java)

```
package com.example.ems.controller;  
  
import com.example.ems.model.Department;  
import com.example.ems.repository.DepartmentRepository;  
import java.util.List;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;  
  
@RestController  
@RequestMapping("/api/departments")  
public class DepartmentController {  
  
    @Autowired  
    private DepartmentRepository departmentRepository;  
  
    @PostMapping
```

```
public Department createDepartment(@RequestBody Department department)
{
    return departmentRepository.save(department);
}
```

@GetMapping

```
public List<Department> getAllDepartments() {
    return departmentRepository.findAll();
}
```

@GetMapping("/{id}")

```
public ResponseEntity<Department> getDepartmentById(@PathVariable Long
id) {
    return departmentRepository.findById(id)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

@PutMapping("/{id}")

```
public ResponseEntity<Department> updateDepartment(@PathVariable Long
id, @RequestBody Department details) {
    return departmentRepository.findById(id).map(dept -> {
        dept.setName(details.getName());
        return ResponseEntity.ok(departmentRepository.save(dept));
    }).orElse(ResponseEntity.notFound().build());
}
```

```

@DeleteMapping("/{id}")

public ResponseEntity<Void> deleteDepartment(@PathVariable Long id) {

    if (departmentRepository.existsById(id)) {

        departmentRepository.deleteById(id);

        return ResponseEntity.ok().build();

    }

    return ResponseEntity.notFound().build();

}
}

```

Employee Controller (EmployeeController.java)

```

package com.example.ems.controller;

import com.example.ems.model.Employee;
import com.example.ems.repository.EmployeeRepository;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController

@RequestMapping("/api/employees")

public class EmployeeController {

```


@Autowired

```
private EmployeeRepository employeeRepository;
```

@PostMapping

```
public Employee createEmployee(@RequestBody Employee employee) {  
    return employeeRepository.save(employee);  
}
```

@GetMapping

```
public List<Employee> getAllEmployees() {  
    return employeeRepository.findAll();  
}
```

@GetMapping("/{id}")

```
public ResponseEntity<Employee> getEmployeeById(@PathVariable Long id)  
{  
    return employeeRepository.findById(id)  
        .map(ResponseEntity::ok)  
        .orElse(ResponseEntity.notFound().build());  
}
```

@PutMapping("/{id}")

```
public ResponseEntity<Employee> updateEmployee(@PathVariable Long id,  
@RequestBody Employee details) {  
    return employeeRepository.findById(id).map(emp -> {  
        emp.setName(details.getName());
```

```

        emp.setEmail(details.getEmail());

        emp.setDepartment(details.getDepartment());

        return ResponseEntity.ok(employeeRepository.save(emp));

    }).orElse(ResponseEntity.notFound().build());
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteEmployee(@PathVariable Long id) {

    if (employeeRepository.existsById(id)) {

        employeeRepository.deleteById(id);

        return ResponseEntity.ok().build();

    }

    return ResponseEntity.notFound().build();

}
}

```

Exercise 5: Defining Query Methods

Entity Named Query Mapping (Employee.java)

```

@javax.persistence.NamedQueries({
    @javax.persistence.NamedQuery(
        name = "Employee.fetchByEmailNamed",
        query = "SELECT e FROM Employee e WHERE e.email = :email"
    )
})

```

Repository Expansion (EmployeeRepository.java)

```

package com.example.ems.repository;

import com.example.ems.model.Employee;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

```

```

import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    List<Employee> findByNameContaining(String keyword);
    Employee findByEmail(String email);

    @Query("SELECT e FROM Employee e LEFT JOIN FETCH e.department WHERE e.name LIKE %:name%")
    List<Employee> searchByNameCustomHql(@Param("name") String name);

    @Query(value = "SELECT * FROM employees WHERE employee_email = :email", nativeQuery = true)
    Employee searchByEmailNative(@Param("email") String email);

    Employee fetchByEmailNamed(@Param("email") String email);
}

```

Exercise 6: Implementing Pagination and Sorting

Repository Modification (EmployeeRepository.java)

```

org.springframework.data.domain.Page<Employee>
findByNameContaining(String keyword,
org.springframework.data.domain.Pageable pageable);

```

Controller Expansion (EmployeeController.java)

```

@GetMapping("/search")
public org.springframework.data.domain.Page<Employee>
searchEmployeesPaged(
    @RequestParam String keyword,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size,
    @RequestParam(defaultValue = "name") String sortBy,
    @RequestParam(defaultValue = "asc") String direction) {

    org.springframework.data.domain.Sort sort =
direction.equalsIgnoreCase("desc") ?
        org.springframework.data.domain.Sort.by(sortBy).descending() :
        org.springframework.data.domain.Sort.by(sortBy).ascending();
    org.springframework.data.domain.Pageable pageable =
org.springframework.data.domain.PageRequest.of(page, size, sort);
    return employeeRepository.findByNameContaining(keyword, pageable);
}

```

Exercise 7: Enabling Entity Auditing

Auditing Setup Class (AuditConfig.java)

```
package com.example.ems.config;

import java.util.Optional;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.domain.AuditorAware;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;

@Configuration
@EnableJpaAuditing
public class AuditConfig {

    @Bean
    public AuditorAware<String> auditorProvider() {
        return () -> Optional.of("System_User");
    }
}
```

Superclass Creation (Auditable.java)

```
package com.example.ems.model;

import java.time.LocalDateTime;
import javax.persistence.Column;
import javax.persistence.EntityListeners;
import javax.persistence.MappedSuperclass;
import lombok.Getter;
import lombok.Setter;
import org.springframework.data.annotation.CreatedBy;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedBy;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
@Getter
@Setter
public abstract class Auditable {

    @CreatedBy
    @Column(name = "created_by", updatable = false)
    private String createdBy;
```

```

@CreatedDate
@Column(name = "created_date", updatable = false)
private LocalDateTime createdDate;

@LastModifiedBy
@Column(name = "last_modified_by")
private String lastModifiedBy;

@LastModifiedDate
@Column(name = "last_modified_date")
private LocalDateTime lastModifiedDate;
}

```

Exercise 8: Creating Projections

Interface-Based Projection (EmployeeProjection.java)

```

package com.example.ems.projection;

import org.springframework.beans.factory.annotation.Value;

public interface EmployeeProjection {

    String getName();

    String getEmail();

    @Value("#{target.department != null ? target.department.name : 'Unassigned'}")
    String getDepartmentName();

}

```

Class-Based Projection DTO (EmployeeDto.java)

```

package com.example.ems.projection;

```

```
import lombok.AllArgsConstructor;
```

```
import lombok.Getter;
```

```
@Getter
```

```
@AllArgsConstructor
```

```
public class EmployeeDto {
```

```
    private final String name;
```

```
    private final String email;
```

```
}
```

Projections Repository Query Methods (EmployeeRepository.java)

```
@Query("SELECT e.name as name, e.email as email FROM Employee e WHERE  
e.department.id = :deptId")
```

```
List<com.example.ems.projection.EmployeeProjection>  
fetchProjectionsByDepartment(@Param("deptId") Long deptId);
```

```
@Query("SELECT new com.example.ems.projection.EmployeeDto(e.name,  
e.email) FROM Employee e WHERE e.department.id = :deptId")
```

```
List<com.example.ems.projection.EmployeeDto>  
fetchClassDtosByDepartment(@Param("deptId") Long deptId);
```

Exercise 9: Customizing Data Source Configuration

Multi-DataSource properties definition (application.properties)

```
spring.datasource.primary.url=jdbc:h2:mem:primarydb;DB_CLOSE_DELAY=-1
```

```
spring.datasource.primary.driverClassName=org.h2.Driver
```

```
spring.datasource.primary.username=sa
```

spring.datasource.primary.password=password

spring.datasource.secondary.url=jdbc:h2:mem:secondarydb;DB_CLOSE_DELAY=-1

spring.datasource.secondary.driverClassName=org.h2.Driver

spring.datasource.secondary.username=sa

spring.datasource.secondary.password=password

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

spring.jpa.hibernate.ddl-auto=update

Configuration Class (DataSourceConfig.java)

```
package com.example.ems.config;
```

```
import javax.persistence.EntityManagerFactory;
```

```
import javax.sql.DataSource;
```

```
import org.springframework.beans.factory.annotation.Qualifier;
```

```
import org.springframework.boot.context.properties.ConfigurationProperties;
```

```
import org.springframework.boot.jdbc.DataSourceBuilder;
```

```
import org.springframework.boot.orm.jpa.EntityManagerFactoryBuilder;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.context.annotation.Primary;
```

```
import org.springframework.data.jpa.repository.config.EnableJpaRepositories;
```

```
import org.springframework.orm.jpa.JpaTransactionManager;
```

```
import org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean;
```

```
import org.springframework.transaction.PlatformTransactionManager;
```

```
@Configuration
```

```
@EnableJpaRepositories(
```

```
    basePackages = "com.example.ems.repository",
```

```
    entityManagerFactoryRef = "primaryEntityManagerFactory",
```

```
    transactionManagerRef = "primaryTransactionManager"
```

```
)
```

```
public class DataSourceConfig {
```

```
    @Primary
```

```
    @Bean(name = "primaryDataSource")
```

```
    @ConfigurationProperties(prefix = "spring.datasource.primary")
```

```
    public DataSource primaryDataSource() {
```

```
        return DataSourceBuilder.create().build();
```

```
    }
```

```
    @Primary
```

```
    @Bean(name = "primaryEntityManagerFactory")
```

```
    public LocalContainerEntityManagerFactoryBean  
    primaryEntityManagerFactory(
```

```
        EntityManagerFactoryBuilder builder,
```

```
        @Qualifier("primaryDataSource") DataSource dataSource) {
```

```
        return builder
```



```

        .dataSource(dataSource)

        .packages("com.example.ems.model")

        .persistenceUnit("primary")

        .build();
    }

    @Primary

    @Bean(name = "primaryTransactionManager")

    public PlatformTransactionManager primaryTransactionManager(

        @Qualifier("primaryEntityManagerFactory") EntityManagerFactory
        entityManagerFactory) {

        return new JpaTransactionManager(entityManagerFactory);
    }

    @Bean(name = "secondaryDataSource")

    @ConfigurationProperties(prefix = "spring.datasource.secondary")

    public DataSource secondaryDataSource() {

        return DataSourceBuilder.create().build();
    }
}

```

Exercise 10: Hibernate-Specific Features

Configuration Optimization Properties (application.properties)

spring.jpa.properties.hibernate.jdbc.batch_size=20

spring.jpa.properties.hibernate.order_inserts=true

spring.jpa.properties.hibernate.order_updates=true

Controller Batch Save Implementation (EmployeeController.java)

```
@org.springframework.transaction.annotation.Transactional

@PostMapping("/batch")

public ResponseEntity<String> saveEmployeesBatch(@RequestBody
List<Employee> employees) {

    int batchSize = 20;

    List<Employee> subList = new ArrayList<>();

    for (int i = 0; i < employees.size(); i++) {

        subList.add(employees.get(i));

        if ((i + 1) % batchSize == 0 || (i + 1) == employees.size()) {

            employeeRepository.saveAll(subList);

            subList.clear();

        }

    }

    return ResponseEntity.ok("Batch processing completed successfully");

}
```